

Day 103: Feature Engineering Basics

1. Day Number

Day 103

2. Topic Name

Creating Useful Features

3. Connection

You already know that machine-learning models learn from **features**.

Today you will learn that useful features do not always have to come directly from the original dataset.

Sometimes we can create new features from existing information:

```
Raw data
  ↓
Transform / combine
  ↓
More useful features
  ↓
Model
```

This process is called **feature engineering**.

4. Important Topics

Raw Feature

A **raw feature** comes directly from the original dataset.

For example:

```
transaction_date
amount
customer_age
```

If the dataset already contains `customer_age`, then age is a raw feature.

Derived Feature

A **derived feature** is created from one or more existing values.

For example:

```
transaction_date → transaction_month
```

or:

```
amount → amount_category
```

Categorical Feature

A **categorical feature** represents groups or categories.

Examples:

```
city = "Bengaluru"  
payment_method = "Card"  
amount_category = "High"
```

These are different from continuous numerical values such as:

```
age = 28  
amount = 1200
```

Date-Derived Feature

Dates contain several pieces of potentially useful information.

From:

```
2026-09-27
```

you could potentially derive:

```
year  
month  
day  
day_of_week
```

You should create only features that make sense for the problem.

Feature Selection Idea

Feature selection simply means deciding:

| Which available features are actually useful for the model?

You do not necessarily need to give every possible column to a model.

For now, feature selection is just a **reasoning process**. We will not use automated feature-selection algorithms.

5. Foundational Notes

Consider this transaction:

Transaction Date	Amount	Customer Age
2026-09-27	2500	32

These are raw values.

But perhaps the exact date itself is less useful than knowing:

```
month = September
```

Maybe the exact amount could also produce a broader category:

```
amount = 2500
      ↓
amount_category = "High"
```

So your dataset could contain both raw and derived information.

The important idea is:

Feature engineering tries to represent the available information in a form that makes useful patterns easier for a model to learn.

6. Why Better Features Can Help a Model

Imagine you want to predict whether customers are likely to make another purchase.

You have:

```
transaction_date
```

The full date contains useful information, but a model may benefit from a simpler feature such as:

```
transaction_month
```

Why?

Perhaps customer behavior changes by month.

For example:

```
November → more purchases
December → more purchases
January  → fewer purchases
```

A month feature makes that pattern easier to represent directly.

Similarly:

```
amount = 75
amount = 850
amount = 5000
```

could potentially become:

```
Low
Medium
High
```

if those categories are meaningful for your problem.

Feature engineering can help because it can:

- make useful patterns easier to see
- simplify raw information
- represent domain knowledge
- convert information into forms a model can use more effectively

But creating **more features does not automatically mean creating better features**.

7. Easy Examples

Example 1: Date → Month

Raw feature:

```
transaction_date = 2026-09-27
```

Derived feature:

```
transaction_month = 9
```

Example 2: Date → Day of Week

Raw feature:

```
transaction_date
```

Derived feature:

```
day_of_week
```

This might be useful if customer behavior differs between weekdays and weekends.

Example 3: Age → Age Group

Raw feature:

```
customer_age = 32
```

Possible derived feature:

```
age_group = "30-39"
```

The exact grouping should depend on the actual problem.

Example 4: Amount → Amount Category

Raw feature:

```
amount = 2500
```

Possible derived feature:

```
amount_category = "High"
```

You must define what counts as:

```
Low  
Medium  
High
```

instead of choosing categories randomly.

8. Problem Statement

Suppose you have this transaction dataset:

Transaction Date	Amount	Customer Age
2026-01-15	300	22
2026-04-10	1800	35
2026-09-25	4200	58

Your task is to suggest a few simple derived features.

Possible directions to think about include:

transaction date → month
amount → amount category
customer age → age group

For each proposed feature, explain:

1. Which raw feature it comes from
2. How you would create it
3. Why it could potentially be useful

Do not train a model.

9. Concepts Used

Today's exercise uses:

- raw features
- derived features
- numerical features
- categorical features
- date-derived features
- feature engineering
- feature selection
- prediction-time availability

The overall idea is:

```
Raw dataset
  ↓
Understand the problem
  ↓
Create sensible derived features
  ↓
Choose useful features
  ↓
Later: give them to a model
```

10. Thought Process

When looking at a raw column, ask:

Step 1: What does this column mean?

For example:

transaction_date

represents when a transaction happened.

Step 2: Does it contain smaller pieces of useful information?

A date contains:

year
month
day
day of week

Perhaps one of those matters more than the complete date.

Step 3: Can a numerical value be represented meaningfully another way?

For example:

amount

could potentially become:

small purchase
medium purchase
large purchase

Step 4: Would the new feature actually help?

Do not create features just because you can.

A feature should have some reasonable connection to the problem.

11. Beginner-Friendly Pseudocode

Conceptually:

```
START
```

```
load transaction data
```

```
for each transaction:
```

```
    read transaction_date
```

```
    read amount
```

```
    read customer_age
```

```
    extract month from transaction_date
```

```
    convert amount into an appropriate category
```

```
    optionally create an age group
```

```
add useful derived features to dataset
```

```
review whether each feature makes sense
```

```
END
```

Notice that feature engineering happens **before model training**.

12. Features Must Be Available at Prediction Time

This is a very important rule.

Suppose you are building a model to predict:

| Will this customer purchase again?

Imagine you create this feature:

```
number_of_purchases_next_month
```

There is a problem.

At the moment you are making the prediction, **next month has not happened yet**.

So that information is unavailable.

Using it during model development would give the model information from the future.

Instead, valid features might include things already known at prediction time:

```
customer_age
```

```
current transaction amount
```

```
past number of purchases
```

```
transaction month
```

A useful question is:

| **Could I know this value at the exact moment I need to make the prediction?**

If the answer is no, it generally should not be used as an input feature for that prediction.

This also helps avoid a common machine-learning problem called **data leakage**, which you will encounter more formally later.

13. Easy Edge Cases

Missing Input

Suppose:

```
transaction_date = missing
```

Then you cannot directly calculate:

```
transaction_month
```

You first need to decide how missing dates should be handled.

Useless Feature

Suppose you create:

```
always_one = 1
```

for every transaction.

It contains no variation.

That feature probably provides little or no useful information to the model.

Redundant Feature

Suppose your dataset contains:

```
customer_age = 30  
customer_age_copy = 30
```

The second feature contains essentially the same information.

More columns do not automatically provide more useful information.

Bad Categories

Suppose you define:

Low = amount below 1000
Medium = amount between 1000 and 2000
High = amount above 2000

Those boundaries should ideally have some reasonable justification.

Arbitrary categories can remove useful detail rather than improve the data.

14. Expected Transformed Dataset

Your final dataset might conceptually have a structure like this:

Transaction Date	Amount	Customer Age	Month	Amount Category	Age Group
2026-01-15	300	22	?	?	?
2026-04-10	1800	35	?	?	?
2026-09-25	4200	58	?	?	?

The original columns are still available, while new columns contain **derived features**.

You do not necessarily need every derived feature in the final model. Later, you can decide which ones are genuinely useful.

15. Hint Only

Start with the easiest column:

Transaction Date

Ask:

| What useful piece can I extract directly from a date?

Then inspect:

Amount

Decide on a simple rule that divides the values into understandable categories such as:

Low
Medium
High

Finally consider:

Customer Age

Ask whether keeping the exact age, creating an age group, or perhaps keeping both would make sense.

The key lesson for Day 103 is:

Raw data

↓

Create meaningful information

↓

Useful features

But every new feature should pass two checks:

Does it make sense for the prediction problem, and will it actually be available when the prediction is made?